

Prolog Lab 6

Fifteen Classic Problems Explained from Scratch

A Complete Beginner's Guide

How to use this guide. Part I teaches every Prolog idea you need, once, in plain language. Part II solves the 15 lab problems. Every problem follows the same eight-part layout: (1) Problem Statement, (2) Prolog Code, (3) Prolog Concepts Used, (4) Line-by-Line Explanation, (5) How Prolog Works Internally, (6) Step-by-Step Execution, (7) Sample Queries and Outputs, (8) Key Takeaways. Each program is also provided as a separate `.pl` file (`p01_factorial.pl` ... `p15_uncle_aunt.pl`).

Contents

I	Prolog Basics You Need for Every Problem	2
1	Running Prolog: Programs, Queries and Answers	2
2	The Building Blocks: Terms	2
3	Facts, Rules, Queries, Predicates	3
4	Unification: How Prolog Matches Things	4
5	How Prolog Executes a Query (The Search Engine)	4
6	Recursion	5
7	Lists	5
8	Arithmetic and Comparison	6
9	Negation and the Anonymous Variable	7
10	Collecting All Answers: <code>findall/3</code> and <code>sort/2</code>	7
11	Quick Reference Tables	7
II	The Fifteen Problems	9

1	Factorial Calculation (Recursion)	9
2	Sum of Even Numbers in a List	12
3	Palindrome Checker	15
4	Maximum Element in a List	18
5	List Length Without Built-in Predicates	20
6	Reverse a List	22
7	Ancestor and Descendant Relations	25
8	Simple Arithmetic Expression Evaluator	28
9	Count Occurrences of an Element	31
10	Find All Siblings	33
11	N-th Element of a List	36
12	Check if List is Sorted	38
13	Check Prime Number	40
14	Fibonacci Series Generator (Non-memoized)	43
15	Family Tree – Uncle/Aunt Relation	46
	Summary of All Problems	50

Part I

Prolog Basics You Need for Every Problem

1 Running Prolog: Programs, Queries and Answers

Prolog is a *logic programming language*. In languages like C or Python you tell the computer *how* to do something, step by step. In Prolog you describe *what is true* (facts and rules), and then you *ask questions*. Prolog searches for the answer itself.

This guide uses **SWI-Prolog**, the most popular free implementation.

1. Save a program in a text file, for example `p01_factorial.pl`.
2. Start SWI-Prolog in a terminal: `swipl p01_factorial.pl` (or start `swipl` and type `[p01_factorial].` or `consult('p01_factorial.pl').`).
3. Prolog shows the prompt `?-`. Type a question (a *query*) and end it with a full stop `.`

```
?- fact(5, F).
F = 120.
```

Definition: The answers Prolog can give

Answer	Meaning
<code>true.</code>	The question is true (and there were no variables to report).
<code>X = value.</code>	The question is true when the variable <code>X</code> has this value.
<code>false.</code>	Prolog searched <i>every</i> possibility and found none. This is a normal answer, <i>not</i> an error.
<code>ERROR: ...</code>	You made a mistake (for example, an unknown predicate or a variable used in arithmetic before it has a value).

Watch out: the `;` prompt

When Prolog gives an answer but still has untried possibilities, it may wait after the answer. Press **Enter** (or `.`) to stop, or press `;` (meaning “or?”) to ask for another answer. For the deterministic examples in this guide, the extra answer is simply `false.` In the sample outputs we show the answers only, without these prompts.

2 The Building Blocks: Terms

Everything in Prolog is a **term**. There are only a few kinds:

Kind	Looks like	Notes
Atom	tom, bob, add, []	Starts with a <i>lowercase</i> letter. A constant name; it only means itself.
Number	0, 42, -3, 2.5	Integers and decimals.
Variable	X, Name, _, _Tmp	Starts with an <i>uppercase</i> letter or underscore. A placeholder that Prolog fills in.
Compound term	parent(tom, bob), add(3, 4)	A name (the <i>functor</i>) followed by arguments in parentheses.
List	[1,2,3], [a,b T], []	An ordered sequence (see Section 7).

Mnemonic

Capital = Changeable. If a word starts with a **C**apital letter it is a variable (it can change).
lowercase = locked constant.

3 Facts, Rules, Queries, Predicates

Definition: Fact

A statement that is unconditionally true. It ends with a full stop.
parent(tom, bob). reads: “Tom is a parent of Bob.”

Definition: Rule

A statement of the form `Head :- Body`. The symbol `:-` is read “if”. The head is true *if* every goal in the body is true. Goals in the body are separated by commas, and a comma means **AND**.
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
reads: “X is a grandparent of Z if X is a parent of some Y and that Y is a parent of Z.”

Definition: Query (Goal)

A question you type at the `?-` prompt, such as `?- parent(tom, bob)`. Prolog tries to prove it from your facts and rules.

Definition: Clause, Predicate, Arity

A **clause** is one fact or one rule. A **predicate** is a group of clauses with the same name and the same number of arguments. The number of arguments is the **arity**. We write a predicate as `name/arity`, for example `fact/2`. Note that `eval/2` and `eval/3` would be two different predicates.

A Prolog program is a database of clauses. Prolog contains no “main” function; you start the computation by asking a query.

	Fact	Rule
Shape	head.	head :- goal1, goal2.
Meaning	Always true	True only if the body is true
Example	male(tom).	uncle(U,C) :- parent(P,C), sibling(U,P), male(U).

4 Unification: How Prolog Matches Things

Unification is Prolog’s one fundamental operation: “make these two terms identical, by giving values to variables if necessary.” It is what the `=` operator does, and it also happens every time Prolog matches a goal against a clause head.

Unify this	With this	Result
<code>tom</code>	<code>tom</code>	Succeeds (identical atoms)
<code>tom</code>	<code>bob</code>	Fails (different atoms)
<code>X</code>	<code>5</code>	Succeeds, <code>X = 5</code>
<code>f(X, 2)</code>	<code>f(1, Y)</code>	Succeeds, <code>X = 1</code> , <code>Y = 2</code>
<code>f(X, X)</code>	<code>f(1, 2)</code>	Fails: <code>X</code> cannot be both 1 and 2
<code>[H T]</code>	<code>[a,b,c]</code>	Succeeds, <code>H = a</code> , <code>T = [b,c]</code>
<code>[H T]</code>	<code>[]</code>	Fails (the empty list has no head)

Two rules to remember:

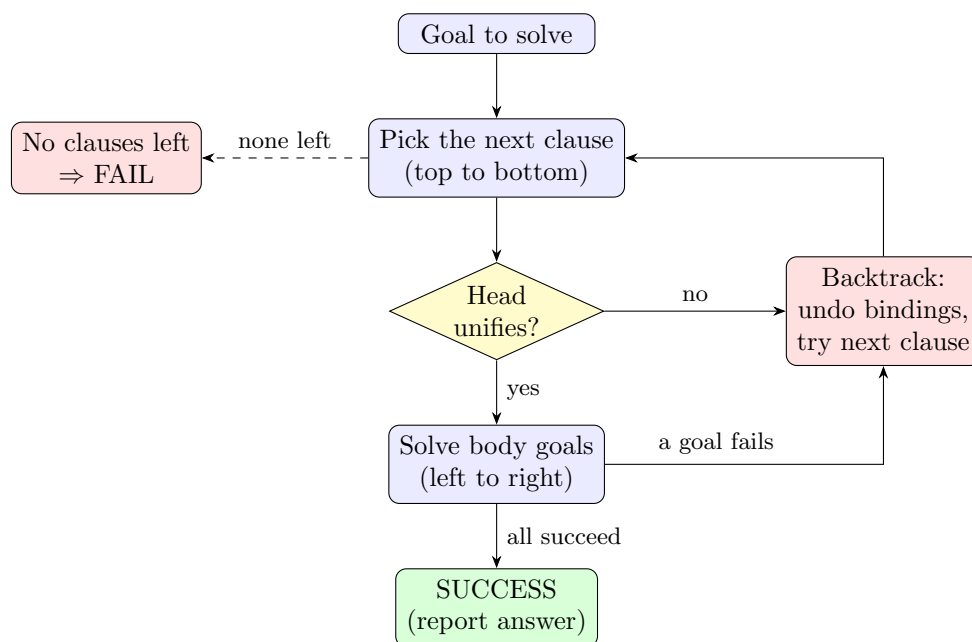
- A variable can be bound to a value, and once bound it **keeps** that value (until Prolog backtracks, see below). You cannot “update” a variable like `x = x + 1` in C.
- If the same variable appears twice in one clause head, both places must receive the same value. Problem 9 uses this deliberately.

5 How Prolog Executes a Query (The Search Engine)

Prolog uses a fixed, mechanical procedure. Learn it once and every program becomes predictable.

Definition: Prolog’s execution rules

1. **Goals are solved left to right.** In `a, b, c` Prolog solves `a` first, then `b`, then `c`.
2. **Clauses are tried top to bottom** in the order they appear in the file.
3. To solve a goal, Prolog finds a clause whose *head unifies* with the goal. If it is a fact, the goal succeeds. If it is a rule, Prolog now must solve the body goals.
4. If a goal **fails**, Prolog **backtracks**: it goes back to the most recent place where it made a choice (a *choice point*), undoes the variable bindings made since then, and tries the next alternative.
5. If no alternatives are left anywhere, the original query answers **false**.



Choice points. When more than one clause could match, Prolog remembers the untried clauses. If the current attempt later fails, or if you press `;`, Prolog resumes from that memory. This automatic try-and-retry is called **backtracking**.

Fresh variables. Each time Prolog uses a clause it makes a *private copy* of the clause with brand-new variables. This is why a recursive rule can call itself: the `N` in one call is a different variable from the `N` in the next call.

6 Recursion

A **recursive** predicate calls itself on a smaller problem. Because Prolog has no loops (`for`, `while`), recursion is how you repeat work. Every correct recursive predicate has two parts:

Pattern: Base case + recursive case

1. **Base case:** the smallest problem, answered directly (usually a fact). It stops the recursion.
2. **Recursive case:** reduces the problem to a smaller one, calls itself, then combines the result.

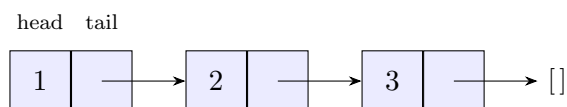
The call stack. Prolog remembers each unfinished call, like a stack of plates. Calls pile up until the base case is reached; then answers are passed back down, one call at a time. In the factorial problem you will see this drawn.

Mnemonic

B-R-C: **B**ase case first, **R**ecurse on something smaller, **C**ombine the result. If you forget the smaller step, the program runs forever (stack overflow).

7 Lists

A list is written in square brackets: `[a,b,c]`. The empty list is `[]`. Internally, a list is a chain: a **Head** (first element) joined to a **Tail** (the rest of the list, which is itself a list). The notation `[H|T]` splits a list into these two parts; the vertical bar `|` is read “followed by”.



Pattern	Matches	Example match with [a,b,c]
[]	only the empty list	no match
[X]	a list of exactly one element	no match (3 elements)
[H T]	any non-empty list	H = a, T = [b,c]
[X,Y T]	a list with at least two elements	X = a, Y = b, T = [c]
[_ T]	non-empty list, ignore the head	T = [b,c]

Pattern: The standard list recursion

```

process([], BaseAnswer). % empty list
process([H|T], Answer) :-
    process(T, AnswerForTail), % solve the smaller list
    combine(H, AnswerForTail, Answer). % use the head + tail answer
  
```

Almost every list problem in this guide (Problems 2–6, 9, 11, 12) is a variation of this shape.

8 Arithmetic and Comparison

Watch out: = does not calculate

In Prolog, `X = 2 + 3` does *not* give 5. It just unifies `X` with the term `2+3`. To calculate, use `is`.

Operator	What it does	Example
=	Unifies two terms (no calculation)	<code>X = 2+3</code> gives <code>X = 2+3</code>
\=	Succeeds if two terms <i>cannot</i> unify	<code>a \= b</code> is true
is	Calculates the right side, unifies result with the left	<code>X is 2+3</code> gives <code>X = 5</code>
==	Arithmetic equal	<code>4 == 2+2</code> is true
==\=	Arithmetic not equal	<code>3 ==\= 2+2</code> is true
< > =< >=	Arithmetic comparison	<code>3 =< 3</code> is true

Useful arithmetic functions: `+` `-` `*` `/`, `mod` (remainder, so `7 mod 2 == 1`), and `max(A,B)`.

Important rules.

- The right side of `is` (and both sides of `<`, `==`, etc.) must contain only numbers or variables that *already have numeric values*. Otherwise Prolog reports an error.
- “Less than or equal” is written `==<`, *not* `<=`.
- The left side of `is` is normally a fresh variable: write `N1 is N - 1`, never `N is N - 1` (that would try to give `N` a second value).

Mnemonic

“= matches, is calculates, == compares.” Also: the arrow of less-or-equal points to the equal sign: =<.

9 Negation and the Anonymous Variable

Negation as failure: \+ Goal succeeds when Prolog *cannot prove* Goal. Read \+ as “it is not provable that”. Problem 13 uses it.

Anonymous variable: a single underscore _ means “some value, but I do not care what.” Every _ is independent of every other _. Using it avoids “singleton variable” warnings for variables that appear only once.

10 Collecting All Answers: findall/3 and sort/2

Normally Prolog gives answers one at a time. The predicate `findall(Template, Goal, List)` runs Goal and *collects every solution* into List. Here, Template says which variable’s value you want to save from each solution.

```
?- findall(X, member(X, [a,b,c]), L).
L = [a, b, c].
```

If there are no solutions, `findall` succeeds with the empty list [].

`sort(List, Sorted)` puts the list in standard order *and removes duplicates*. Problem 10 relies on this.

11 Quick Reference Tables

Problem you hit	Typical cause	Fix
Unknown procedure foo/2	Misspelled name or wrong arity	Check spelling and argument count
Arguments are not sufficiently instantiated	A variable used in is or < has no value yet	Make sure the variable is bound earlier in the body
Program never stops / stack overflow	Missing base case, or the recursive step does not shrink the problem	Add a base case; shrink the argument
Unexpected false	Order of goals wrong, or a guard such as <code>N > 0</code> fails	Test sub-goals one at a time
Singleton variable warning	A variable appears only once in a clause	Rename it to _ or fix a typo

Want to watch Prolog think? Type `trace.` then your query. Prolog prints every `Call`, `Exit`, `Redo` and `Fail` step. Type `notrace.` to stop. The step-by-step sections in Part II mimic what this trace shows.

Mnemonic

Prolog’s four port words: C-E-R-F (“**C**all, **E**xit, **R**edo, **F**ail”): Call starts a goal, Exit means it succeeded, Redo means “try another way”, Fail means no way worked.

Part II

The Fifteen Problems

Problem 1: Factorial Calculation (Recursion)

1. Problem Statement

Write a Prolog program to calculate the factorial of a number using recursion.

Definition: Factorial

$n! = n \times (n - 1) \times \cdots \times 2 \times 1$, and by definition $0! = 1$.

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. Notice that $n! = n \times (n - 1)!$. This self-reference is exactly what recursion needs.

2. Prolog Code

File: p01_factorial.pl

```
1 fact(0, 1). % base case: 0! = 1
2 fact(N, F) :- % recursive case
3     N > 0,
4     N1 is N - 1,
5     fact(N1, F1),
6     F is N * F1.
```

3. Prolog Concepts Used

- **Fact** (line 1) and **rule** (lines 2–6).
- **Recursion** with a base case and a recursive case.
- **Arithmetic with is** and the comparison **>**.
- **Guard**: a test ($N > 0$) that decides whether a clause is allowed to apply.
- **Fresh variables per call**: each recursive call has its own N , $N1$, F , $F1$.

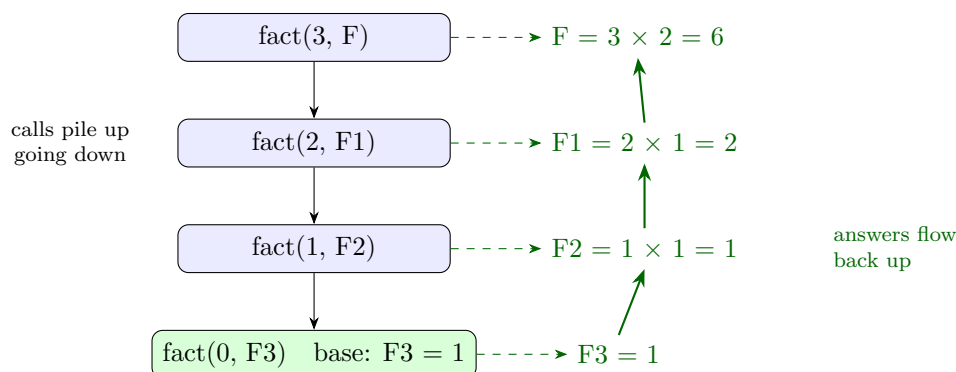
4. Line-by-Line Explanation

- Line 1** `fact(0, 1).` A fact: “the factorial of 0 is 1.” This is the **base case**; the recursion stops here.
- Line 2** `fact(N, F) :-` The head of the **recursive rule**: “the factorial of N is F , *if* the goals below are all true.” N is the input, F the output.
- Line 3** `N > 0,` A guard. It makes sure we only recurse for positive numbers. (Without it, a negative input would recurse forever.)
- Line 4** `N1 is N - 1,` Calculates $N - 1$ and stores it in the new variable $N1$. This is the *smaller problem*.
- Line 5** `fact(N1, F1),` The recursive call: “let $F1$ be the factorial of $N1$.”

Line 6 `F is N * F1`. Once `F1` is known, compute $N \times F1$ and give it to `F`. This uses the identity $n! = n \times (n-1)!$.

5. How Prolog Works Internally

1. **Matching the goal to clauses.** For the goal `fact(3, F)`, Prolog tries clause 1 first: can `fact(0, 1)` unify with `fact(3, F)`? No, because $3 \neq 0$. So Prolog moves on to clause 2, whose head `fact(N, F)` unifies with anything, giving $N = 3$.
2. **Solving the body left to right.** $3 > 0$ is true. `N1 is 3-1` binds $N1 = 2$. Then comes the recursive goal `fact(2, F1)`. Prolog must finish it before it can look at the last line.
3. **Fresh variables.** The new call uses a renamed copy of the clause, so you can picture `N_2`, `F_2`, `N1_2`, `F1_2`. The values from the first call are not disturbed.
4. **The stack builds up.** The calls for 3,2,1,0 wait for each other. Only `fact(0, F)` can be answered immediately (clause 1).
5. **The answer is built on the way back.** Each waiting call now finishes its last line `F is N * F1` and hands its result to the caller.
6. **Why the guard matters.** The goal `fact(-1, F)` matches clause 2, but $-1 > 0$ fails, so Prolog answers **false**. Without the guard, the sequence $-1, -2, -3, \dots$ would never reach 0 and Prolog would run out of memory.



6. Step-by-Step Execution / Query Flow

Query: `?- fact(3, F).`

```
CALL fact(3, F)
  clause 1 fact(0,1)? 3 vs 0 -> no match
  clause 2 fact(N,F): N = 3
    3 > 0 -> true
    N1 is 3 - 1 -> N1 = 2
    CALL fact(2, F1)
      clause 1? 2 vs 0 -> no. clause 2: N = 2
        2 > 0 -> true
        N1 is 2 - 1 -> N1 = 1
        CALL fact(1, F2)
          clause 1? 1 vs 0 -> no. clause 2: N = 1
            1 > 0 -> true
            N1 is 1 - 1 -> N1 = 0
            CALL fact(0, F3)
              clause 1 fact(0,1) matches -> F3 = 1 (EXIT)
              F2 is 1 * 1 -> F2 = 1 (EXIT)
            F1 is 2 * 1 -> F1 = 2 (EXIT)
          F is 3 * 2 -> F = 6 (EXIT)
```

Answer: $F = 6$.

Worked numerical problem. Compute `fact(5, F)` by hand: the calls go $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$. The base case gives 1. Coming back: $1 \times 1 = 1$, $2 \times 1 = 2$, $3 \times 2 = 6$, $4 \times 6 = 24$, $5 \times 24 = \mathbf{120}$.

7. Sample Queries and Outputs

```
?- fact(5, F).  
F = 120.  
  
?- fact(0, F).  
F = 1.  
  
?- fact(10, F).  
F = 3628800.  
  
?- fact(-3, F).  
false.  
  
?- fact(4, 24). % checking an answer instead of computing one  
true.
```

Key Takeaways

- Base case first (`fact(0,1)`), then the recursive rule.
- The recursive call must work on a *smaller* number (`N1 is N-1`).
- The result is combined *after* the recursive call returns.
- The guard `N > 0` protects against endless recursion.

Mnemonic

“Zero is one, then times down.” $0! = 1$, and each level multiplies by the answer from below.

Problem 2: Sum of Even Numbers in a List

1. Problem Statement

Define a predicate `sum_even(List, Sum)` that returns the sum of all even numbers in a given list.

2. Prolog Code

File: `p02_sum_even.pl`

```

1 sum_even([], 0). % empty list: sum is 0
2 sum_even([H|T], Sum) :- % H is even: add it
3     H mod 2 == 0,
4     sum_even(T, Rest),
5     Sum is Rest + H.
6 sum_even([H|T], Sum) :- % H is odd: skip it
7     H mod 2 =\= 0,
8     sum_even(T, Sum).
```

3. Prolog Concepts Used

- **List recursion** with `[]` as the base case and `[H|T]` to take the list apart.
- **mod** (remainder after division): a number is even when `H mod 2 == 0`.
- **Arithmetic comparison** `==` (equal) and `=\=` (not equal).
- **Two rules for one pattern**, separated by mutually exclusive guards (even versus odd).

4. Line-by-Line Explanation

Line 1 `sum_even([], 0).` The sum of an empty list is 0. Base case.

Line 2 `sum_even([H|T], Sum) :-` Rule for a non-empty list. `H` is the first element, `T` the rest. This clause handles the case where the first element is even.

Line 3 `H mod 2 == 0,` Guard: the remainder of `H` divided by 2 is 0, so `H` is even.

Line 4 `sum_even(T, Rest),` Recursively compute the sum of evens in the tail and call it `Rest`.

Line 5 `Sum is Rest + H.` Since `H` is even, add it to the tail’s sum.

Line 6 `sum_even([H|T], Sum) :-` Second rule for a non-empty list: the case where the first element is not even.

Line 7 `H mod 2 =\= 0,` Guard: the remainder is not 0, so `H` is odd.

Line 8 `sum_even(T, Sum).` Skip `H`. The sum of the whole list equals the sum of the tail, so the *same* variable `Sum` is passed along unchanged.

5. How Prolog Works Internally

- For a non-empty list, *both* clause 2 and clause 3 have heads that unify. Prolog tries clause 2 first (top to bottom) and leaves a **choice point** for clause 3.

- If H is odd, the guard on line 3 fails. Prolog **backtracks** to the choice point and tries clause 3, whose guard succeeds.
- If H is even, clause 2 succeeds. Clause 3 stays on the shelf; if it were ever tried, its guard $H \bmod 2 \neq 0$ would fail, so no wrong second answer can appear.
- The recursion shrinks the list by one element each time and finally reaches $[],$ matched by line 1.

Watch out: why both clauses need a guard

Suppose line 7 were missing. For the query `sum_even([2], S),` Prolog would give $S = 2$; then, on pressing `;`, clause 3 would also match and give $S = 0$, which is a wrong second answer. The guard makes the two cases *mutually exclusive*.

6. Step-by-Step Execution / Query Flow

Query: `?- sum_even([1,2,3,4], S).`

```
sum_even([1,2,3,4], S)
  clause 1: [] vs [1,2,3,4] -> no
  clause 2: H=1, T=[2,3,4]; 1 mod 2 := 0 ? 1 := 0 -> fails
  clause 3: H=1; 1 mod 2 \= 0 ? true
    sum_even([2,3,4], S)
      clause 2: H=2; 2 mod 2 := 0 -> true
        sum_even([3,4], Rest)
          clause 2: H=3; 1 := 0 -> fails
          clause 3: H=3; odd -> true
            sum_even([4], Rest)
              clause 2: H=4; 0 := 0 -> true
                sum_even([], Rest4) -> clause 1: Rest4 = 0
                Sum is 0 + 4 -> Rest = 4
              Sum is 4 + 2 -> S = 6
    Sum is 4 + 2 -> S = 6
Answer: S = 6.
```

Worked numerical problem. For $[1, 2, 3, 4, 5, 6]$: the evens are 2, 4, 6, so the sum is $2 + 4 + 6 = 12$. Prolog computes it as $0 + 6 = 6$, then $6 + 4 = 10$, then $10 + 2 = 12$ (odd elements pass the value unchanged).

7. Sample Queries and Outputs

```
?- sum_even([1,2,3,4,5,6], S).
S = 12.

?- sum_even([1,3,5], S).
S = 0.

?- sum_even([], S).
S = 0.

?- sum_even([10,-4,7], S).
S = 6.
```

Key Takeaways

- Typical list recursion: empty list is the base case; otherwise take $[H|T]$.
- Use *two* rules when the head needs a different treatment, and make their guards opposite.
- Skipping an element means passing the *same* result variable to the recursive call.

Mnemonic

“Even: add it. Odd: let it go.”

Problem 3: Palindrome Checker

1. Problem Statement

Write a predicate `palindrome(List)` that succeeds if the given list is a palindrome (reads the same forwards and backwards, for example `[r,a,c,e,c,a,r]`).

2. Prolog Code

File: `p03_palindrome.pl`

```

1 palindrome(List) :-
2     reverse_list(List, List). % a palindrome equals its own reverse
3
4 reverse_list(List, Reversed) :-
5     reverse_acc(List, [], Reversed).
6 reverse_acc([], Acc, Acc).
7 reverse_acc([H|T], Acc, Reversed) :-
8     reverse_acc(T, [H|Acc], Reversed).
```

3. Prolog Concepts Used

- **Simplest idea:** a list is a palindrome exactly when it *equals its own reverse*.
- **Unification as an equality test:** using the same variable twice forces both places to match.
- **Helper predicates and accumulators** (to reverse a list).
- **Reusing** one predicate inside another.

4. Line-by-Line Explanation

- Line 1** `palindrome(List) :-` “List is a palindrome if the goal below holds.”
- Line 2** `reverse_list(List, List).` Reverse List and demand that the reversed result be List again. Writing the *same variable* in both argument positions is the equality test.
- Line 3** A blank line (ignored by Prolog; it only separates the helper predicates).
- Line 4** `reverse_list(List, Reversed) :-` A user-friendly wrapper: reverse List to give Reversed.
- Line 5** `reverse_acc(List, [], Reversed).` Starts the real work with an *accumulator* (a collecting list) that begins as the empty list [].
- Line 6** `reverse_acc([], Acc, Acc).` When the input is used up, the accumulator *is* the reversed list, so the two last arguments must be equal.
- Line 7** `reverse_acc([H|T], Acc, Reversed) :-` Otherwise take the head H...
- Line 8** `reverse_acc(T, [H|Acc], Reversed).` ...put H on the *front* of the accumulator, and continue with the tail. Because each new head is placed in front, the order flips.

5. How Prolog Works Internally

- In `palindrome([a,b,a])`, the call on line 2 becomes `reverse_list([a,b,a],[a,b,a])`. So the *third* argument of `reverse_acc` is already known: `[a,b,a]`.
- Prolog builds the reversed list in the accumulator (second argument) while walking down the input. It does not use the known third argument until the base case.
- At the base case (line 6), the head `reverse_acc([], Acc, Acc)` forces the accumulator and the third argument to **unify**. If they are the same list, unification succeeds; if not, it fails, and Prolog answers **false**.
- After a failure Prolog backtracks, but no other clause can match (line 7 needs a non-empty list), so the final answer is **false**.

6. Step-by-Step Execution / Query Flow

Query: `?- palindrome([a,b,a]).`

```
palindrome([a,b,a])
-> reverse_list([a,b,a], [a,b,a])
-> reverse_acc([a,b,a], [], [a,b,a]) H=a T=[b,a]
-> reverse_acc([b,a], [a], [a,b,a]) H=b T=[a]
-> reverse_acc([a], [b,a], [a,b,a]) H=a T=[]
-> reverse_acc([], [a,b,a], [a,b,a]) base clause: Acc=[a,b,a]
                                     third arg [a,b,a] -> unify OK
Answer: true.
```

Now a failing example, `?- palindrome([a,b,c]).`

```
reverse_acc([a,b,c], [], [a,b,c])
reverse_acc([b,c], [a], [a,b,c])
reverse_acc([c], [b,a], [a,b,c])
reverse_acc([], [c,b,a], [a,b,c]) base clause: Acc=[c,b,a]
                                     third arg [a,b,c] -> [c,b,a] vs [a,b,c]
                                     unification FAILS -> no other clause
Answer: false.
```

7. Sample Queries and Outputs

```
?- palindrome([r,a,c,e,c,a,r]).
true.

?- palindrome([1,2,3]).
false.

?- palindrome([]).
true.

?- palindrome([x]).
true.
```

Key Takeaways

- Break a problem into an idea you already know: `palindrome` = “equals its reverse”.
- The same variable in two argument positions makes Prolog test equality by unification.
- The accumulator trick (collect while walking down) reverses a list efficiently.

Mnemonic

“Reverse it and compare” (Rotor, level, and 12321 are palindromes).

Problem 4: Maximum Element in a List

1. Problem Statement

Write a predicate `max_list(List, Max)` to find the largest number in a list using recursion.

2. Prolog Code

File: `p04_max_list.pl`

```
1 max_list([X], X). % one element: it is the maximum
2 max_list([H|T], Max) :-
3     max_list(T, MaxTail), % maximum of the rest
4     Max is max(H, MaxTail). % bigger of H and that maximum
```

3. Prolog Concepts Used

- **List recursion** where the base case is a *one-element* list, not an empty one.
- **Pattern** `[X]` (a list with exactly one element).
- The built-in arithmetic function `max(A, B)` used with `is`.
- **Solving the tail first**, then combining with the head.

Note. SWI-Prolog also has a library predicate with the same name. A definition written in your own file takes priority, so the program above works as expected.

4. Line-by-Line Explanation

- Line 1** `max_list([X], X).` A list with just one element: that element is the maximum. Notice how the same variable `X` appears twice.
- Line 2** `max_list([H|T], Max) :-` Rule for longer lists: `H` is the first element, `T` the rest.
- Line 3** `max_list(T, MaxTail),` Find the maximum of the rest of the list.
- Line 4** `Max is max(H, MaxTail).` The overall maximum is the larger of the head and the tail’s maximum.

5. How Prolog Works Internally

- The empty list `[]` has no maximum, so there is deliberately no clause for it. The goal `max_list([], M)` simply fails.
- For `max_list([5], M)`, line 1 matches: `[X]` unifies with `[5]` giving `X = 5`, then the second argument `X` unifies with `M`, giving `M = 5`.
- For a list with two or more elements, line 1 cannot match (the list is not of the form `[X]`), so Prolog uses line 2 and recurses on the tail.
- For a one-element tail, line 2 *also* matches `[H|T]` with `T = []`. This leaves a harmless choice point: if tried, `max_list([], _)` immediately fails.
- The answer is built *while returning*: each level compares its own head with the answer from below.

6. Step-by-Step Execution / Query Flow

Query: `?- max_list([3,9,2], M).`

```
max_list([3,9,2], M)
  clause 1 [X] vs [3,9,2]: tail is not [] -> no match
  clause 2: H=3, T=[9,2]
    max_list([9,2], MaxTail)
      clause 2: H=9, T=[2]
        max_list([2], MaxTail2)
          clause 1: X=2 -> MaxTail2 = 2
          Max is max(9, 2) -> 9
          Max is max(3, 9) -> 9
Answer: M = 9.
```

Worked numerical problem. `max_list([4,1,7,3], M)`: tail results come back as 3; then $\max(7, 3) = 7$; then $\max(1, 7) = 7$; then $\max(4, 7) = 7$.

7. Sample Queries and Outputs

```
?- max_list([3,9,2,7], M).
M = 9.

?- max_list([5], M).
M = 5.

?- max_list([-4,-8,-1], M).
M = -1.

?- max_list([], M).
false.
```

Key Takeaways

- Choose the base case that makes sense: a maximum needs at least one element.
- Pattern `[X]` means “exactly one element”.
- `is` together with `max/2` avoids writing separate “bigger or smaller” rules.

Mnemonic

“Head versus the champion of the tail.” Each level lets the head challenge the best of the rest.

Problem 5: List Length Without Built-in Predicates

1. Problem Statement

Define a recursive rule to calculate the length of a list, without using built-in predicates such as `length/2`.

2. Prolog Code

File: `p05.list_length.pl`

```
1 list_length([], 0). % empty list has length 0
2 list_length(_|T, N) :-
3     list_length(T, N1), % length of the tail
4     N is N1 + 1. % plus one for the head
```

3. Prolog Concepts Used

- **List recursion** (base case `[]`, recursive case `_|T`).
- **Anonymous variable** `_` for the head, since its value does not matter.
- **Counting on the way back** with `is`.

Why the name `list_length`? `length/2` is a built-in (“system”) predicate of SWI-Prolog and cannot be redefined. A new name avoids the error `No permission to modify static procedure`.

4. Line-by-Line Explanation

Line 1 `list_length([], 0).` The empty list has length 0. Base case.

Line 2 `list_length(_|T, N) :-` A non-empty list. We only care that a head exists, not what it is, so we write `_`. `T` is the remaining list.

Line 3 `list_length(T, N1),` Find the length of the tail and call it `N1`.

Line 4 `N is N1 + 1.` The whole list is one element longer than its tail.

5. How Prolog Works Internally

- Matching `_|T` with `[a,b,c]` binds `T = [b,c]`; the anonymous variable matches `a` and is then forgotten.
- Each recursive call removes one element, so the list gets shorter until it becomes `[]`, which matches line 1.
- Only then can `N is N1 + 1` run for the deepest call (because `N1` must have a number first). The results are added up while the stack unwinds.
- Clause selection is easy: `[]` can only match line 1, and a non-empty list can only match line 2. There is never more than one possible clause, so no needless backtracking occurs.

6. Step-by-Step Execution / Query Flow

Query: `?- list_length([a,b,c], N).`

```
list_length([a,b,c], N)
  line 2: T = [b,c]
  list_length([b,c], N1)
    line 2: T = [c]
    list_length([c], N1b)
      line 2: T = []
      list_length([], N1c)
        line 1: N1c = 0
        N1b is 0 + 1 -> 1
        N1 is 1 + 1 -> 2
        N is 2 + 1 -> 3
Answer: N = 3.
```

Worked numerical problem. For $[x,y,z,w]$: base 0; then $0+1 = 1$; $1+1 = 2$; $2+1 = 3$; $3+1 = 4$. The length is 4. Note that $[1, [2,3], 4]$ has length **3**: the inner list counts as a single element.

7. Sample Queries and Outputs

```
?- list_length([a,b,c], N).
N = 3.

?- list_length([], N).
N = 0.

?- list_length([1,[2,3],4], N).
N = 3.

?- list_length([a,b], 2).
true.
```

Key Takeaways

- Counting = “length of tail plus one”.
- Use `_` for values you do not need.
- Avoid names that clash with built-in predicates (`length/2`).

Mnemonic

“One for the head, the tail does the rest.”

Problem 6: Reverse a List

1. Problem Statement

Define a predicate `reverse_list(Input, Output)` using recursion. For example, reversing `[1,2,3]` gives `[3,2,1]`.

2. Prolog Code

File: `p06.reverse_list.pl`

```
1 reverse_list(Input, Output) :-
2     reverse_acc(Input, [], Output). % start with an empty accumulator
3 reverse_acc([], Acc, Acc). % input used up: accumulator is the answer
4 reverse_acc([H|T], Acc, Output) :-
5     reverse_acc(T, [H|Acc], Output). % move H onto the front of Acc
```

3. Prolog Concepts Used

- **Accumulator:** an extra argument that collects the answer step by step.
- **Helper predicate** (`reverse_acc/3`) with a friendly wrapper (`reverse_list/2`).
- **List construction** with `[H|Acc]`: placing an element at the front of a list.
- **List recursion**, as in the earlier problems.

Pattern: Accumulator

Add an extra argument that starts as an empty result. At every step, move one piece of the input into the accumulator. When the input is empty, the accumulator holds the finished answer, and it is simply copied to the output.

4. Line-by-Line Explanation

- Line 1** `reverse_list(Input, Output) :-` The public predicate asked for in the problem.
- Line 2** `reverse_acc(Input, [], Output).` Delegates to the helper, giving it an *empty accumulator* `[]` to start with.
- Line 3** `reverse_acc([], Acc, Acc).` Base case: no input left. Whatever was collected in the accumulator is the answer, so the 2nd and 3rd arguments are made equal. This is where `Output` finally gets its value.
- Line 4** `reverse_acc([H|T], Acc, Output) :-` Input is non-empty: split it into head `H` and tail `T`.
- Line 5** `reverse_acc(T, [H|Acc], Output).` Continue with the tail, and put `H` at the *front* of the accumulator. Since the first element is stacked first and later elements go in front of it, the order is flipped.

5. How Prolog Works Internally

- The variable `Output` is *passed unchanged* through every recursive call. It stays unbound all the way down, and gets its value only when line 3 unifies it with the accumulator.
- Therefore there is **no work to do on the way back up**: the answer is complete at the bottom. (This style is called tail recursion and is memory-friendly.)
- Clause choice is clear-cut: `[]` only matches line 3, and a non-empty list only matches line 4.
- `[H|Acc]` *builds* a new list with `H` in front. It is the same notation used to take lists apart, but now it is used on the “constructing” side.

	Accumulator version (used here)	Naive version (append at the end)
Idea	Move each head onto the front of a growing list	Reverse the tail, then stick the head at the end
Needs <code>append</code> ?	No	Yes (extra concept)
Speed	One step per element	Repeated copying: much slower for long lists
Beginner difficulty	Slightly unusual idea, but short code	Very natural idea, but needs <code>append/3</code>

6. Step-by-Step Execution / Query Flow

Query: `?- reverse_list([1,2,3], R).`

```
reverse_list([1,2,3], R)
-> reverse_acc([1,2,3], [], R) H=1 T=[2,3] new Acc = [1]
-> reverse_acc([2,3], [1], R) H=2 T=[3] new Acc = [2,1]
-> reverse_acc([3], [2,1], R) H=3 T=[] new Acc = [3,2,1]
-> reverse_acc([], [3,2,1], R) base clause: Acc = [3,2,1], so R = [3,2,1]
Answer: R = [3,2,1].
```

Worked numerical problem. Reverse `[a,b,c,d]`: the accumulator grows as `[] → [a] → [b,a] → [c,b,a] → [d,c,b,a]`. The answer is `[d,c,b,a]`.

7. Sample Queries and Outputs

```
?- reverse_list([1,2,3], R).
R = [3, 2, 1].

?- reverse_list([], R).
R = [].

?- reverse_list([a], R).
R = [a].

?- reverse_list([1,2,3], [3,2,1]).
true.
```

Key Takeaways

- Accumulator = an extra argument that carries partial results.
- The base case copies the accumulator to the output.

- `[H|Acc]` puts an element in front of a list; this naturally reverses the order.

Mnemonic

“Pile of plates.” Take plates from the top of one pile and place them onto a new pile: the new pile is reversed.

Problem 7: Ancestor and Descendant Relations

1. Problem Statement

Given `parent/2` facts, define `ancestor/2` and `descendant/2` recursively.

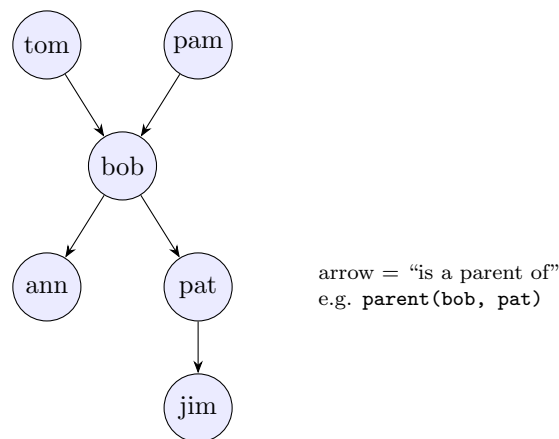
2. Prolog Code

File: `p07_ancestor_descendant.pl`

```

1 parent(tom, bob).
2 parent(pam, bob).
3 parent(bob, ann).
4 parent(bob, pat).
5 parent(pat, jim).
6
7 ancestor(X, Y) :- % direct parent
8     parent(X, Y).
9 ancestor(X, Y) :- % parent of an ancestor chain
10    parent(X, Z),
11    ancestor(Z, Y).
12
13 descendant(X, Y) :- % X is a child of Y
14    parent(Y, X).
15 descendant(X, Y) :- % X is a child of someone who descends from Y
16    parent(Z, X),
17    descendant(Z, Y).
```

The family used in the facts:



3. Prolog Concepts Used

- **Facts as a database** (`parent/2`).
- **Recursion on relationships** (not on lists or numbers): an ancestor is a parent, or a parent of an ancestor.
- **Several clauses for one predicate**: Prolog tries them in order.
- **Backtracking to find all answers**: pressing `;` makes Prolog hunt for the next solution.
- **Intermediate variable** (`Z`) linking two goals.

4. Line-by-Line Explanation

Line 1–5

The `parent/2` facts. `parent(tom, bob)` means “tom is a parent of bob”.

Line 7–8

`ancestor(X, Y) :- parent(X, Y).` **Base case:** a parent is an ancestor.

Line 9–11

`ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).` **Recursive case:** X is an ancestor of Y if X is a parent of some person Z, and Z is an ancestor of Y. The chain is followed one generation at a time.

Line 13–14

`descendant(X, Y) :- parent(Y, X).` **Base case:** a child is a descendant of its parent. Notice the swapped order: Y is the parent of X.

Line 15–17

`descendant(X, Y) :- parent(Z, X), descendant(Z, Y).` **Recursive case:** X is a descendant of Y if X’s parent Z is a descendant of Y.

5. How Prolog Works Internally

- **Two clauses, one goal.** For any `ancestor` goal Prolog first tries line 8 (direct parent). If this fails (or when you ask for more answers), it tries lines 9–11. Both kinds of answer are found.
- **Variable search.** In `parent(X, Z)`, Z is unbound, so Prolog scans the `parent` facts from top to bottom and tries each one. If the rest of the body fails for a fact, it backtracks to the next fact. This automatic search is the heart of Prolog.
- **The recursion ends** because every step moves one generation down the family tree and the tree is finite. Eventually `ancestor(Z, Y)` reaches either a direct `parent` fact or a person with no children (failure).
- **Order matters.** The recursive call is placed *after* `parent(X, Z)`. If you wrote `ancestor(Z, Y)` first (“left recursion”), Prolog would call `ancestor` again with the same variables forever.

	<code>ancestor(X, Y)</code>	<code>descendant(X, Y)</code>
Meaning	X is above Y in the tree	X is below Y in the tree
Base case	<code>parent(X, Y)</code>	<code>parent(Y, X)</code>
Recursive step	<code>parent(X,Z), ancestor(Z,Y)</code> (go down)	<code>parent(Z,X), descendant(Z,Y)</code> (go up)
Relationship	<code>descendant(A,B)</code> is true exactly when <code>ancestor(B,A)</code> is true.	

6. Step-by-Step Execution / Query Flow

Query 1: `?- ancestor(tom, jim).`

```

ancestor(tom, jim)
  clause 1: parent(tom, jim)? no such fact -> fail
  clause 2: parent(tom, Z) -> first match: Z = bob
    ancestor(bob, jim)
      clause 1: parent(bob, jim)? no -> fail
      clause 2: parent(bob, Z) -> first match: Z = ann

```

```

    ancestor(ann, jim)
    clause 1: parent(ann, jim)? no
    clause 2: parent(ann, Z)? ann has no children -> fail
backtrack: next fact parent(bob, Z) -> Z = pat
    ancestor(pat, jim)
    clause 1: parent(pat, jim)? YES
Answer: true.

```

Query 2: `?- ancestor(X, jim).` (who are all of Jim’s ancestors?) Prolog first finds `X = pat` (clause 1: the fact `parent(pat, jim)`). On pressing `;` it uses clause 2 and walks the facts in order: `tom` (via `bob`, then `pat`), `pam` (via `bob`, then `pat`), skips `ann`’s branch (`bob-ann` leads nowhere), then `bob` (via `pat`). `Pat`’s own child `jim` leads nowhere.

Worked example (descendants of tom). `descendant(X, tom)`: first `X = bob` (direct child). Then via recursion: `ann` and `pat` (children of `bob`, who descends from `tom`), then `jim` (child of `pat`).

7. Sample Queries and Outputs

```

?- ancestor(tom, jim).
true.

?- ancestor(X, jim).
X = pat ;
X = tom ;
X = pam ;
X = bob.

?- descendant(X, tom).
X = bob ;
X = ann ;
X = pat ;
X = jim.

?- descendant(tom, jim).
false.

```

(The `;` after each answer is what you type to request the next one.)

Key Takeaways

- A recursive relation = “directly related, or related through someone who is.”
- Base case first and non-recursive; recursive goal *last* in the body.
- Prolog finds all answers by backtracking; use `;` to see them.
- `descendant` mirrors `ancestor` with the `parent` arguments swapped.

Mnemonic

“**P**arent, or **p**arent of a **p**arent...” (A-N-C: **A**ncessor = **p**arent **N**ext **C**hain.)

Problem 8: Simple Arithmetic Expression Evaluator

1. Problem Statement

Given a term like `add(3, mul(2, 4))`, evaluate the expression.

2. Prolog Code

File: `p08_expression_evaluator.pl`

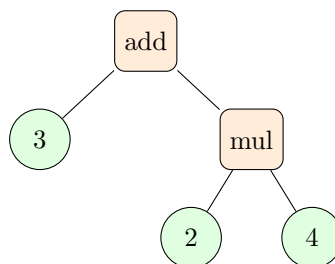
```

1 eval(N, N) :- % a plain number is its own value
2   number(N).
3 eval(add(A, B), R) :-
4   eval(A, RA),
5   eval(B, RB),
6   R is RA + RB.
7 eval(sub(A, B), R) :-
8   eval(A, RA),
9   eval(B, RB),
10  R is RA - RB.
11 eval(mul(A, B), R) :-
12  eval(A, RA),
13  eval(B, RB),
14  R is RA * RB.
15 eval(div(A, B), R) :-
16  eval(A, RA),
17  eval(B, RB),
18  R is RA / RB.
```

3. Prolog Concepts Used

- **Compound terms** such as `add(3, mul(2,4))`: structures with a name and arguments, which can be nested.
- **Pattern matching on structure**: the clause heads `eval(add(A,B), R)` match only terms starting with `add`.
- **Recursion on a tree**: evaluate both sub-expressions, then combine them.
- **Type test `number/1`**: succeeds if its argument is a number.
- **`is` for the actual calculation.**

An expression is a tree. Evaluating means working from the leaves (numbers) up to the root:



4. Line-by-Line Explanation

Line 1–2

`eval(N, N) :- number(N).` A number evaluates to itself. The same variable `N` in both positions means “the value equals the input”. The guard `number(N)` stops it matching `add(...)` and the like. **Base case.**

Line 3–6

The `add` rule: the head only matches a term shaped like `add(A, B)`. Line 4 evaluates `A` to `RA`, line 5 evaluates `B` to `RB` (both are recursive calls), and line 6 adds the two results.

Line 7–10

The same pattern for `sub` (subtraction), using `-`.

Line 11–14

The same pattern for `mul` (multiplication), using `*`.

Line 15–18

The same pattern for `div` (division), using `/`.

5. How Prolog Works Internally

- **Structure drives clause selection.** For `eval(mul(2,4), R)`, Prolog tries clause 1: the head `eval(N, N)` would bind `N = mul(2,4)` and `R = mul(2,4)`, but then `number(N)` fails, so *both bindings are undone* (backtracking). Clauses for `add` and `sub` do not match the name `mul`. The `mul` clause matches and binds `A = 2, B = 4`.
- **Nested expressions solve themselves.** When `A` is itself an expression (like `mul(2,4)`), the call `eval(A, RA)` simply starts the same process again, one level deeper.
- **Why the arguments are evaluated before `is`.** The right-hand side of `is` must contain only numbers. By evaluating the parts first into `RA` and `RB`, which hold plain numbers, the final line is always safe.
- **Division.** `8/4` gives the integer 2, but `10/4` gives 2.5. Dividing by zero raises an error (this is a mistake in the query, not a normal `false`).

6. Step-by-Step Execution / Query Flow

Query: `?- eval(add(3, mul(2,4)), R).`

```
eval(add(3, mul(2,4)), R)
  clause 1: N = add(...); number(add(...))? no -> undo, try next
  clause 2 (add): A = 3, B = mul(2,4)
    eval(3, RA)
      clause 1: number(3) yes -> RA = 3
    eval(mul(2,4), RB)
      clause 1: number(mul(2,4))? no
      clauses for add, sub: wrong name -> no match
      clause for mul: A = 2, B = 4
        eval(2, RA2) -> 2
        eval(4, RB2) -> 4
        R2 is 2 * 4 -> 8
      RB = 8
    R is 3 + 8 -> 11
Answer: R = 11.
```

Worked numerical problem. `div(mul(6,4), sub(7,4))`: left side $6 \times 4 = 24$; right side $7 - 4 = 3$; result $24/3 = 8$.

7. Sample Queries and Outputs

```
?- eval(add(3, mul(2, 4)), R).  
R = 11.  
  
?- eval(sub(10, add(2, 3)), R).  
R = 5.  
  
?- eval(div(mul(6, 4), sub(7, 4)), R).  
R = 8.  
  
?- eval(7, R).  
R = 7.  
  
?- eval(div(10, 4), R).  
R = 2.5.
```

Key Takeaways

- A nested term is a tree; recursion visits every node.
- Each clause head acts like a pattern for one operator.
- Evaluate sub-expressions first, then calculate with **is**.
- To support a new operator, just add one more clause.

Mnemonic

“Leaves first, then climb.” Numbers are leaves; operators combine what their children return.

Problem 9: Count Occurrences of an Element

1. Problem Statement

Write a predicate `count_elem(List, Elem, Count)` that counts how many times `Elem` appears in `List`.

2. Prolog Code

File: `p09_count_elem.pl`

```

1 count_elem([], _, 0). % nothing to count in an empty list
2 count_elem([E|T], E, Count) :- % head equals the element we look for
3     count_elem(T, E, Rest),
4     Count is Rest + 1.
5 count_elem([H|T], E, Count) :- % head is something else
6     H \= E,
7     count_elem(T, E, Count).
```

3. Prolog Concepts Used

- **List recursion** with three cases: empty list, head matches, head does not match.
- **Repeated variable in a clause head** (`[E|T]` and `E`): matching by unification.
- `\=` (“cannot be unified”) as a guard.
- **Counting on the way back** with `is`.

4. Line-by-Line Explanation

- Line 1** `count_elem([], _, 0).` An empty list contains the element zero times, whatever the element is (`_`).
- Line 2** `count_elem([E|T], E, Count) :-` The *same* variable `E` is used for the head of the list and for the second argument. This clause therefore applies only when the head **equals** the element we are counting.
- Line 3** `count_elem(T, E, Rest),` Count the occurrences in the tail.
- Line 4** `Count is Rest + 1.` The head was a match, so add one.
- Line 5** `count_elem([H|T], E, Count) :-` Case where the head may be something else.
- Line 6** `H \= E,` Guard: `H` and `E` are different, so this head does not count.
- Line 7** `count_elem(T, E, Count).` Skip the head; the count of the whole list equals the count of the tail.

5. How Prolog Works Internally

- **Repeated variables mean “must be equal”.** To match the goal `count_elem([b,a], a, C)` with line 2’s head, Prolog binds `E` to the first argument’s head `b`, then must unify `E` with the second argument `a`. Since `b` and `a` differ, unification fails and line 2 is rejected *before* its body starts.

- Prolog then moves to line 5, whose guard $b \neq a$ is true, so it skips the head.
- If the head does match (for $[a, \dots]$ and a), line 2 applies. Line 5 would also unify with the head, but its guard $a \neq a$ fails, so no duplicate answer is produced.
- The base case (line 1) stops the recursion once the list is empty.

Operator	Meaning	Typical use
$=$ / $\neq$	Unifies / cannot unify (works on any terms)	Comparing atoms and lists, as in this problem
$==$ / $\neq$	Equal / not equal <i>as numbers</i> (needs numbers)	Arithmetic tests such as $H \bmod 2 == 0$

6. Step-by-Step Execution / Query Flow

Query: `?- count_elem([a,b,a], a, C).`

```
count_elem([a,b,a], a, C)
  clause 2: E = a (from head) and E = a (2nd arg) -> match; T = [b,a]
    count_elem([b,a], a, Rest)
      clause 2: E = b vs a -> mismatch, rejected
      clause 3: H = b; b \neq a true; T = [a]
        count_elem([a], a, Rest2)
          clause 2: match; T = []
            count_elem([], a, Rest3) -> clause 1: Rest3 = 0
              Count2 is 0 + 1 -> 1
            (clause 3 returns the same value) Rest = 1
          Count is 1 + 1 -> 2
        Answer: C = 2.
```

Worked numerical problem. `count_elem([a,b,a,c,a], a, C)`: positions 1, 3, 5 match; the answer is $1 + 1 + 1 = 3$.

7. Sample Queries and Outputs

```
?- count_elem([a,b,a,c,a], a, C).
C = 3.

?- count_elem([a,b,c], z, C).
C = 0.

?- count_elem([], a, C).
C = 0.

?- count_elem([1,2,1,1], 1, 3).
true.
```

Key Takeaways

- Using one variable in two places of a clause head forces equality.
- $\neq$ is the opposite test: “cannot be made equal”.
- Two competing clauses should have conditions that exclude each other.

Mnemonic

“Match: plus one. Miss: pass on.”

Problem 10: Find All Siblings

1. Problem Statement

Given `parent/2` facts, write a rule to find all siblings of a person using `findall/3`.

2. Prolog Code

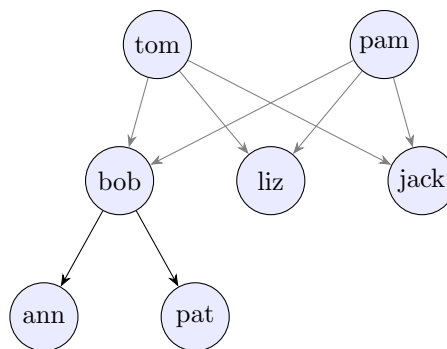
File: `p10_find_siblings.pl`

```

1 parent(tom, bob).
2 parent(pam, bob).
3 parent(tom, liz).
4 parent(pam, liz).
5 parent(tom, jack).
6 parent(pam, jack).
7 parent(bob, ann).
8 parent(bob, pat).
9
10 sibling(X, Y) :- % same parent, but not the same person
11     parent(P, X),
12     parent(P, Y),
13     X \= Y.
14
15 find_siblings(Person, Siblings) :-
16     findall(S, sibling(Person, S), AllSiblings),
17     sort(AllSiblings, Siblings). % sort also removes duplicates

```

The family used in the facts (arrows mean “is a parent of”):



3. Prolog Concepts Used

- **Rule with a shared variable:** `P` links the two `parent` goals (same parent).
- `\=` to exclude the person himself or herself.
- `findall/3`: collects every solution of a goal in a list.
- `sort/2`: orders a list and removes duplicates.
- **Backtracking:** `findall` makes Prolog explore all possibilities.

4. Line-by-Line Explanation

Line 1–8

The `parent/2` facts. Tom and Pam are the parents of Bob, Liz and Jack; Bob is the parent of Ann and Pat.

Line 10 `sibling(X, Y) :- “X and Y are siblings if...”`

Line 11 `parent(P, X), ...some P is a parent of X, and`

Line 12 `parent(P, Y), ...the same P is a parent of Y, and`

Line 13 `X \= Y. ...X and Y are not the same person (otherwise everybody would be their own sibling).`

Line 15 `find_siblings(Person, Siblings) :-` The predicate asked for in the problem.

Line 16 `findall(S, sibling(Person, S), AllSiblings),` Collect every value of `S` for which `sibling(Person, S)` holds, into the list `AllSiblings`.

Line 17 `sort(AllSiblings, Siblings).` Remove duplicates (and sort alphabetically) to produce the final list.

5. How Prolog Works Internally

- **How findall works.** Prolog runs the goal. Every time it succeeds, `findall` copies the template value (`S`) into a result list, then *deliberately forces a failure* so that Prolog backtracks and finds the next solution. When no solutions remain, the list is complete and returned. If there were no solutions at all, the result is `[]`.
- **Why duplicates appear.** In this family, Bob and Liz share *two* parents. The sibling relationship is proved once through `P = tom` and once through `P = pam`. So `findall` finds `liz` twice (and `jack` twice).
- **Why sort/2 is used.** It removes these repeated entries, giving each sibling exactly once.
- **Why \= is on the last line.** By the time it runs, both `X` and `Y` have values, so the test is meaningful. Placed earlier, with `Y` still unbound, it would not work as intended.
- Someone with no recorded parents (`tom`) has no siblings by this definition, and `findall` returns `[]`.

6. Step-by-Step Execution / Query Flow

Query: `?- find_siblings(bob, L).` Prolog first runs `findall(S, sibling(bob, S), All):`

```
sibling(bob, S):
  parent(P, bob) -> first match: P = tom
    parent(tom, Y) tries each fact whose first argument is tom:
      Y = bob -> bob \= bob fails (not collected)
      Y = liz -> bob \= liz ok -> S = liz COLLECT liz
      Y = jack -> ok -> S = jack COLLECT jack
    backtrack: parent(P, bob) -> next match: P = pam
      parent(pam, Y)
        Y = bob -> fails
        Y = liz -> COLLECT liz
        Y = jack -> COLLECT jack
      no more parents of bob.
All = [liz, jack, liz, jack]
```

Then `sort(All, Siblings)` removes duplicates and sorts: `Siblings = [jack, liz]`.

Worked example. Siblings of `ann`: her parent is `bob`; `bob`’s other child is `pat` (`ann` herself is excluded by `\=`). Result `[pat]`.

7. Sample Queries and Outputs

```
?- find_siblings(bob, L).  
L = [jack, liz].  
  
?- findall(S, sibling(bob, S), L). % without sort: duplicates remain  
L = [liz, jack, liz, jack].  
  
?- find_siblings(ann, L).  
L = [pat].  
  
?- find_siblings(tom, L). % no parent facts for tom  
L = [].
```

Key Takeaways

- `findall(Template, Goal, List)` = “give me all solutions of `Goal` as a list”.
- Use `\=` to exclude the person from his or her own siblings.
- Use `sort/2` to remove duplicates produced when a relationship can be proved in several ways.
- `findall` never fails: no solutions simply give `[]`.

Mnemonic

“Same parent, different person.” Sibling = *Shared* parent + *Different* child (S-D). And: `findall` = find ALL.

Problem 11: N-th Element of a List

1. Problem Statement

Write a predicate `nth_element(N, List, Elem)` that retrieves the N-th element of a list (1-based indexing: the first element is number 1).

2. Prolog Code

File: `p11_nth_element.pl`

```
1 nth_element(1, [H|_], H). % the 1st element is the head
2 nth_element(N, [_|T], Elem) :-
3     N > 1,
4     N1 is N - 1, % one step closer to the front
5     nth_element(N1, T, Elem). % ask the same question about the tail
```

3. Prolog Concepts Used

- **Recursion on a counter and a list at the same time:** the number goes down while the list gets shorter.
- **Patterns** `[H|_]` (take the head) and `[_|T]` (discard the head).
- **Arithmetic** with `is` and a guard with `>`.
- **Failure as an answer:** asking for a position that does not exist simply fails.

4. Line-by-Line Explanation

- Line 1** `nth_element(1, [H|_], H).` Base case: the 1st element of a list is its head. The same variable `H` in the list pattern and in the third position means `Elem` is the head.
- Line 2** `nth_element(N, [_|T], Elem) :-` Recursive case: throw away the head (`_`) and keep the tail `T`.
- Line 3** `N > 1,` Guard: only recurse if we have not yet reached position 1. This also rejects 0 and negative positions.
- Line 4** `N1 is N - 1,` After removing one element, the wanted element is one position earlier.
- Line 5** `nth_element(N1, T, Elem).` Ask the same question on the shorter list with the smaller number. The answer `Elem` is passed straight through.

5. How Prolog Works Internally

- For `nth_element(3, [a,b,c,d], E)`, line 1 is tried first. Its first argument is the number 1, which does not unify with 3, so it is rejected immediately. Line 2 matches.
- Every recursive call removes the head and decrements `N`. When the counter reaches 1, line 1 matches and `Elem` receives the head of whatever list remains.
- **The answer needs no calculation on the way back:** `Elem` is the same variable all the way down, and it is bound at the bottom.

- **Out of range.** For `nth_element(5, [a,b], E)` the list shrinks to `[]` while `N` is still bigger than 1. Neither clause matches `[]` (both need a head), so the goal fails and Prolog answers **false**.
- **Limitation.** The guard `N > 1` needs `N` to have a numeric value. Asking `nth_element(N, [a,b,c], c)` (“where is `c`?”) raises an error because `N` is unbound.

6. Step-by-Step Execution / Query Flow

Query: `?- nth_element(3, [a,b,c,d], E).`

```
nth_element(3, [a,b,c,d], E)
  clause 1: 1 vs 3 -> no match
  clause 2: N = 3, T = [b,c,d]
    3 > 1 -> true; N1 is 3 - 1 -> 2
    nth_element(2, [b,c,d], E)
      clause 1: 1 vs 2 -> no match
      clause 2: N = 2, T = [c,d]
        2 > 1 -> true; N1 is 2 - 1 -> 1
        nth_element(1, [c,d], E)
          clause 1: 1 = 1 ok; [H|_] = [c,d] -> H = c; E = c
Answer: E = c.
```

Worked numerical problem. 4th element of `[10,20,30,40,50]`: remove 10 (now position 3), remove 20 (position 2), remove 30 (position 1): the head is **40**.

7. Sample Queries and Outputs

```
?- nth_element(3, [a,b,c,d], E).
E = c.

?- nth_element(1, [x,y], E).
E = x.

?- nth_element(4, [10,20,30,40,50], E).
E = 40.

?- nth_element(5, [a,b], E).
false.

?- nth_element(0, [a,b], E).
false.
```

Key Takeaways

- Count down while walking down the list: position N in a list is position $N - 1$ in its tail.
- The base case position is 1 (1-based indexing).
- A guard `N > 1` avoids endless searching and rejects invalid positions.
- Running off the end of the list leads to failure, not to an error.

Mnemonic

“Drop one, count one.” Every time you discard a head, lower the counter by one.

Problem 12: Check if List is Sorted

1. Problem Statement

Write a predicate `sorted(List)` that succeeds only if the list is sorted in ascending order.

2. Prolog Code

File: `p12_sorted.pl`

```
1 sorted([]). % empty list is sorted
2 sorted([_]). % one element is sorted
3 sorted([X, Y|T]) :- % compare the first two elements
4     X =< Y,
5     sorted([Y|T]). % then check the rest
```

3. Prolog Concepts Used

- **Looking at two elements at once** with the pattern `[X, Y|T]`.
- **Multiple base cases** (empty list and single-element list).
- **Arithmetic comparison** `=<`.
- **Predicates that only answer yes or no** (no output variable).

4. Line-by-Line Explanation

Line 1 `sorted([])`. An empty list is sorted (nothing can be out of order).

Line 2 `sorted([_])`. A one-element list is sorted. The element’s value is irrelevant, so we use `_`.

Line 3 `sorted([X, Y|T]) :-` A list with at least two elements: first element `X`, second `Y`, everything after is `T`.

Line 4 `X =< Y`, The first two elements must be in order. (`=<` allows equal values, so `[2,2]` counts as sorted.)

Line 5 `sorted([Y|T])`. Check the rest of the list, starting from `Y`. Because `Y` is kept as the new first element, the next comparison is `Y` versus the element after it. Every neighbouring pair gets checked.

5. How Prolog Works Internally

- **Why line 2 is needed.** The pattern `[X, Y|T]` requires *two* elements. Without line 2, the final one-element list `[5]` would match no clause and the whole check would fail even for a perfectly sorted list.
- **Early exit on failure.** As soon as one pair is out of order, `X =< Y` fails. No other clause can rescue it (a list with two or more elements only matches line 3), so Prolog answers **false** immediately without looking at the rest.
- **Rebuilding the list.** `[Y|T]` on line 5 builds a fresh list by attaching `Y` in front of `T`. This effectively moves the “window” of two elements forward by one position.
- Each call works on a list that is one element shorter, so the recursion ends at line 2.

6. Step-by-Step Execution / Query Flow

Success: `?- sorted([1,2,5]).`

```
sorted([1,2,5])
  clause 3: X=1, Y=2, T=[5]; 1 <= 2 true
    sorted([2,5])
      clause 3: X=2, Y=5, T=[]; 2 <= 5 true
        sorted([5])
          clause 2 matches [] -> true
Answer: true.
```

Failure: `?- sorted([1,3,2]).`

```
sorted([1,3,2])
  clause 3: X=1, Y=3, T=[2]; 1 <= 3 true
    sorted([3,2])
      clause 3: X=3, Y=2, T=[]; 3 <= 2 FALSE -> fail
      no other clause matches a 2-element list
Answer: false.
```

List	Pairs compared	Result
[1,2,2,5,9]	$1 \leq 2$, $2 \leq 2$, $2 \leq 5$, $5 \leq 9$	true
[4,6,3]	$4 \leq 6$ ok, $6 \leq 3$ fails	false
[7]	none	true

7. Sample Queries and Outputs

```
?- sorted([1,2,2,5,9]).
true.

?- sorted([1,3,2]).
false.

?- sorted([]).
true.

?- sorted([7]).
true.
```

Key Takeaways

- A list is sorted if *every pair of neighbours* is in order.
- The pattern `[X, Y|T]` peeks at two elements; pass `[Y|T]` on.
- Provide base cases for both the empty list and the one-element list.
- For a duplicates-not-allowed check, change `<=` to `<`.

Mnemonic

“Each neighbour pair must be fair.” (Left $\leq$ right, all the way down.)

Problem 13: Check Prime Number

1. Problem Statement

Define `is_prime(N)` that checks if `N` is a prime using basic divisor checking.

Definition: Prime number

A prime is a whole number greater than 1 whose only divisors are 1 and itself. Examples: 2, 3, 5, 7, 11. Non-examples: 1 (not greater than 1), 4 ($= 2 \times 2$), 9 ($= 3 \times 3$).

2. Prolog Code

File: `p13_is_prime.pl`

```

1 is_prime(N) :-
2     N > 1, % 0, 1 and negatives are not prime
3     \+ has_divisor(N, 2). % no divisor found, starting from 2
4
5 has_divisor(N, D) :- % D divides N exactly
6     D * D =< N,
7     N mod D == 0.
8 has_divisor(N, D) :- % otherwise try the next D
9     D * D =< N,
10    D1 is D + 1,
11    has_divisor(N, D1).
```

3. Prolog Concepts Used

- **Negation as failure** (`\+`): “there is no divisor”.
- **Helper predicate** that searches for a divisor.
- **Counting upwards** (`D1 is D + 1`) in a recursive search.
- `mod` and `==` to test divisibility.
- **Stopping condition** `D * D =< N`: only try divisors up to the square root of `N`.

4. Line-by-Line Explanation

Line 1 `is_prime(N) :-` “`N` is prime if...”

Line 2 `N > 1,` ...it is greater than 1 (this rules out 1, 0 and negative numbers), and

Line 3 `\+ has_divisor(N, 2).` ...it is *not* the case that some divisor exists, starting the search at 2.

Line 4 `has_divisor(N, D) :-` “`N` has a divisor, starting the search from `D`, if...”

Line 5 `D * D =< N,` ...`D` has not gone past the square root of `N`, and

Line 6 `N mod D == 0.` ...`D` divides `N` exactly (remainder 0). **Found a divisor.**

Line 7 `has_divisor(N, D) :-` Second way to have a divisor: it is found further on.

Line 8 `D * D =< N,` We are still within the square-root limit.

Line 9 `D1 is D + 1`, Move to the next candidate divisor.

Line 10 `has_divisor(N, D1)`. Keep searching with the next candidate.

5. How Prolog Works Internally

- **How `\+` works.** Prolog runs `has_divisor(N, 2)`. If this *succeeds*, `\+` fails (so `is_prime` fails: not prime). If it *fails* after trying everything, `\+` succeeds (prime). Variable bindings made inside a `\+` are never kept.
- **Why only up to the square root.** If $N = a \times b$, at least one of a, b is $\leq \sqrt{N}$. So if no divisor exists up to $\sqrt{N}$, none exists at all. The test $D \times D \leq N$ is the same as $D \leq \sqrt{N}$ but uses only whole-number arithmetic.
- **Two clauses, one search.** Clause 1 (line 4) succeeds when D divides N . If it fails, Prolog backtracks into clause 2 (line 7), which tries $D+1$. When $D \times D > N$, *both* clauses fail at their first goal, which means “no divisor found” and the search ends with failure.
- The recursion terminates because D grows by 1 each time while N stays fixed, so $D * D$ eventually exceeds N .

N	What the search does	Result
1	fails $N > 1$ at once	not prime
2	$2 \times 2 = 4 > 2$: no divisor candidates	prime
4	$D = 2$: $4 \bmod 2 = 0$, divisor found	not prime
9	$D = 2$: remainder 1; $D = 3$: $9 \bmod 3 = 0$	not prime
7	$D = 2$: remainder 1; $D = 3$: $9 > 7$, stop	prime

6. Step-by-Step Execution / Query Flow

Query: `?- is_prime(7).`

```
is_prime(7)
  7 > 1 -> true
  \+ has_divisor(7, 2) (prove has_divisor(7,2); \+ succeeds only if it fails)
    has_divisor(7, 2):
      clause 1: 2*2 = 4 =< 7 true; 7 mod 2 = 1, 1 := 0 false -> fail
      clause 2: 4 =< 7 true; D1 = 3
        has_divisor(7, 3):
          clause 1: 3*3 = 9 =< 7 false -> fail
          clause 2: 9 =< 7 false -> fail
        fails
      fails
    has_divisor failed everywhere -> \+ succeeds
Answer: true.
```

Query: `?- is_prime(15).`

```
has_divisor(15, 2): 4 =< 15, 15 mod 2 = 1 -> clause 1 fails
  clause 2: D1 = 3 -> has_divisor(15, 3):
    clause 1: 9 =< 15 true; 15 mod 3 = 0 true -> SUCCESS (divisor 3 found)
  \+ has_divisor(...) therefore fails -> is_prime(15) fails.
Answer: false.
```

Worked numerical problem. Is 29 prime? $D = 2, 3, 4, 5$ are tested ($4, 9, 16, 25 \leq 29$): $29 \bmod 2 = 1$, $29 \bmod 3 = 2$, $29 \bmod 4 = 1$, $29 \bmod 5 = 4$, none is 0. Then $D = 6$: $36 > 29$, so the search stops: **29 is prime.**

7. Sample Queries and Outputs

```
?- is_prime(7).  
true.  
  
?- is_prime(15).  
false.  
  
?- is_prime(2).  
true.  
  
?- is_prime(1).  
false.  
  
?- is_prime(29).  
true.
```

Key Takeaways

- `\+` Goal succeeds exactly when Goal cannot be proved.
- A helper that counts upward ($D, D+1, \dots$) searches for a divisor.
- $D \times D \leq N$ is a cheap way to stop at the square root.
- `is_prime` needs a number given as input; it is not designed to generate primes.

Mnemonic

“Prime = no divisor up to the root.” (P-R-D: Prime Rejects any Divisor.)

Problem 14: Fibonacci Series Generator (Non-memoized)

1. Problem Statement

Write a predicate `fib(N, F)` to generate the N-th Fibonacci number recursively.

Definition: Fibonacci numbers

$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2).$
 The sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

2. Prolog Code

File: `p14_fibonacci.pl`

```

1 fib(0, 0). % base case 1
2 fib(1, 1). % base case 2
3 fib(N, F) :-
4     N > 1,
5     N1 is N - 1,
6     N2 is N - 2,
7     fib(N1, F1),
8     fib(N2, F2),
9     F is F1 + F2.
```

3. Prolog Concepts Used

- **Two base cases** (`fib(0,0)` and `fib(1,1)`).
- **Two recursive calls** in one rule (*tree recursion*).
- **Intermediate variables** `N1`, `N2`, `F1`, `F2`.
- **Guard** `N > 1` to keep the recursive clause away from the base cases.

4. Line-by-Line Explanation

Line 1 `fib(0, 0).` Base case: the 0th Fibonacci number is 0.

Line 2 `fib(1, 1).` Base case: the 1st Fibonacci number is 1.

Line 3 `fib(N, F) :-` Recursive rule for all larger N.

Line 4 `N > 1,` Guard: this rule applies only when N is at least 2.

Line 5 `N1 is N - 1,` Position just before N.

Line 6 `N2 is N - 2,` Position two before N.

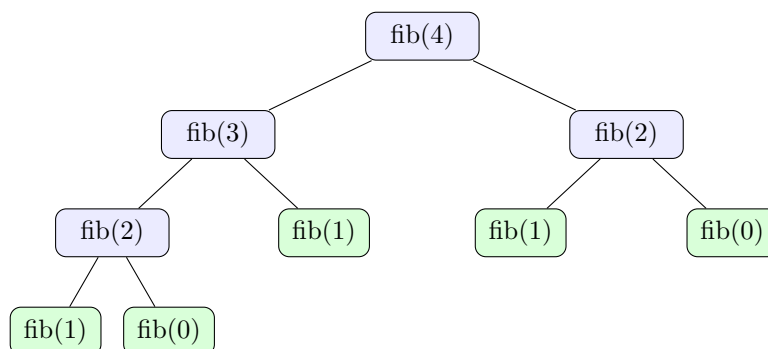
Line 7 `fib(N1, F1),` Compute F_{n-1} recursively.

Line 8 `fib(N2, F2),` Compute F_{n-2} recursively.

Line 9 `F is F1 + F2.` The answer is their sum.

5. How Prolog Works Internally

- **Tree-shaped execution.** The body has *two* recursive goals, so each call spawns two more calls. Prolog finishes line 7 completely (the whole left subtree) before starting line 8.
- **Fresh variables everywhere.** Each call has its own N1, N2, F1, F2, so the many simultaneous calls never interfere.
- **Why the guard.** For `fib(1, F)`, line 2 gives `F = 1`. The head of line 3 would also unify, but `1 > 1` fails, so no wrong second answer appears. Without the guard, the program would also recurse into negative numbers forever.
- **“Non-memoized” means repeated work.** Prolog does not remember results it already computed. In the picture below, `fib(2)` is calculated twice, and `fib(1)` three times, even when computing only `fib(4)`.



Green nodes are base cases. Results: `fib(1)=1`, `fib(0)=0`, `fib(2)=1`, `fib(3)=2`, `fib(4)=3`.

Computing	Number of calls to fib
<code>fib(5)</code>	15
<code>fib(10)</code>	177
<code>fib(20)</code>	21 891
<code>fib(30)</code>	2 692 537

The number of calls roughly multiplies by 1.6 for each step in N. This is why this simple definition becomes very slow for large N (try `fib(35, F)` and `fib(40, F)` to feel it). It is, however, the simplest and clearest definition, which is what the problem asks for.

6. Step-by-Step Execution / Query Flow

Query: `?- fib(4, F).`

```

fib(4, F)
  clause 3: N = 4; 4 > 1; N1 = 3; N2 = 2
  fib(3, F1)
    N = 3; N1 = 2; N2 = 1
    fib(2, Fa)
      N = 2; N1 = 1; N2 = 0
      fib(1, Fb) -> clause 2: Fb = 1
      fib(0, Fc) -> clause 1: Fc = 0
      Fa is 1 + 0 -> 1
      fib(1, Fd) -> clause 2: Fd = 1
      F1 is 1 + 1 -> 2
    fib(2, F2)
      N1 = 1; N2 = 0

```

```
fib(1, ...) -> 1; fib(0, ...) -> 0
F2 is 1 + 0 -> 1
F is 2 + 1 -> 3
Answer: F = 3.
```

Worked numerical problem. Build the sequence bottom-up to check: $F_2 = 1 + 0 = 1$, $F_3 = 1 + 1 = 2$, $F_4 = 2 + 1 = 3$, $F_5 = 3 + 2 = 5$, $F_6 = 5 + 3 = 8$, $F_7 = 8 + 5 = \mathbf{13}$.

7. Sample Queries and Outputs

```
?- fib(0, F).
F = 0.

?- fib(1, F).
F = 1.

?- fib(5, F).
F = 5.

?- fib(7, F).
F = 13.

?- fib(10, F).
F = 55.
```

Key Takeaways

- Two base cases for the first two terms; one recursive rule for the rest.
- Two recursive calls give a call *tree*, not a single chain.
- Without memoization the same sub-problems are recomputed, which is simple but slow for large N.
- Always keep the guard $N > 1$ so that the rules do not overlap.

Mnemonic

“Each number is the sum of the previous two”: 0, 1, then add the last two.

Problem 15: Family Tree – Uncle/Aunt Relation

1. Problem Statement

Extend `parent/2` to define `uncle/2` or `aunt/2` using `sibling/2`. (This solution defines *both*.)

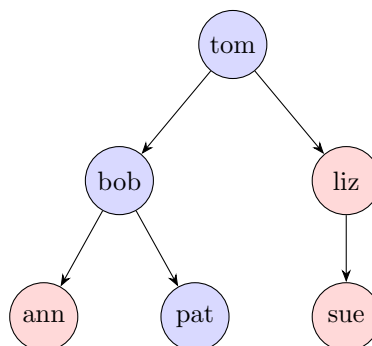
2. Prolog Code

File: `p15_uncle_aunt.pl`

```

1 parent(tom, bob).
2 parent(tom, liz).
3 parent(bob, ann).
4 parent(bob, pat).
5 parent(liz, sue).
6
7 male(tom).
8 male(bob).
9 male(pat).
10 female(liz).
11 female(ann).
12 female(sue).
13
14 sibling(X, Y) :-
15     parent(P, X),
16     parent(P, Y),
17     X \= Y.
18
19 uncle(U, Child) :- % U is a male sibling of Child's parent
20     parent(P, Child),
21     sibling(U, P),
22     male(U).
23
24 aunt(A, Child) :- % A is a female sibling of Child's parent
25     parent(P, Child),
26     sibling(A, P),
27     female(A).
```

The family (arrows mean “is a parent of”; blue = male, pink = female):



3. Prolog Concepts Used

- **Facts for parents and genders** (`male/1`, `female/1`).
- **Building new relations from old ones:** `uncle` uses `parent` and `sibling`.

- **A rule with several goals** joined by `,` (AND).
- **Variables as connectors** between goals (P).
- **Filtering with a fact** (`male(U)`) after finding candidates.

4. Line-by-Line Explanation

Line 1–5

The `parent/2` facts. Tom is the parent of Bob and Liz; Bob of Ann and Pat; Liz of Sue.

Line 7–12

The gender facts: `male/1` for tom, bob, pat; `female/1` for liz, ann, sue.

Line 14–17

`sibling(X, Y)`: they have a common parent P and are different people (same rule as Problem 10).

Line 19 `uncle(U, Child) :-` “U is an uncle of Child if...”

Line 20 `parent(P, Child), ...` P is a parent of Child, and

Line 21 `sibling(U, P), ...` U is a sibling of that parent, and

Line 22 `male(U) . ...` U is male.

Line 24–27

`aunt(A, Child)` is the same rule, but ends with `female(A)`.

5. How Prolog Works Internally

- **Rules calling rules.** `uncle` calls `sibling`, which in turn calls `parent` twice. Each is solved by the same unification-and-backtracking process, with variables passed from one goal to the next.
- **Goal order narrows the search.** `parent(P, Child)` comes first, so P becomes known when `sibling(U, P)` runs. This makes the search efficient: Prolog only looks for the siblings of one specific person.
- **Filtering last.** `male(U)` (or `female(A)`) is a quick check on a person already found. If it fails, Prolog backtracks into `sibling` to look for another candidate.
- **Definition scope.** These rules cover uncles and aunts by blood (siblings of a parent). They do not include an uncle by marriage, since no `married` facts exist.
- **Same shape, different filter.** `uncle` and `aunt` differ only in the last goal.

	<code>uncle(U, Child)</code>	<code>aunt(A, Child)</code>
Step 1	<code>parent(P, Child)</code> : find a parent	<code>parent(P, Child)</code> : find a parent
Step 2	<code>sibling(U, P)</code> : sibling of the parent	<code>sibling(A, P)</code> : sibling of the parent
Step 3	<code>male(U)</code>	<code>female(A)</code>

6. Step-by-Step Execution / Query Flow

Query: `?- uncle(bob, sue).`

```
uncle(bob, sue)
  parent(P, sue) -> P = liz (fact on line 5)
  sibling(bob, liz)
    parent(P2, bob) -> P2 = tom
    parent(tom, liz) -> yes (fact on line 2)
    bob \= liz -> true
  male(bob) -> yes (fact on line 8)
Answer: true.
```

Query: `?- aunt(A, ann).` (who is Ann’s aunt?)

```
aunt(A, ann)
  parent(P, ann) -> P = bob
  sibling(A, bob)
    parent(P2, A) first tries A = bob (tom’s child): then bob \= bob fails
    next candidate: A = liz; parent(tom, bob) true; liz \= bob true
  female(liz) -> yes
Answer: A = liz.
```

Query: `?- uncle(U, ann).` Ann’s parent is Bob; Bob’s only sibling is Liz, but `male(liz)` fails. No other candidate exists, so the answer is **false**.

Worked example. Reading the family picture: Liz is the sister of Bob, so she is an aunt of Ann and Pat (Bob’s children). Bob is the brother of Liz, so he is an uncle of Sue (Liz’s child). This agrees with the answers below.

7. Sample Queries and Outputs

```
?- uncle(bob, sue).
true.

?- aunt(liz, ann).
true.

?- aunt(A, C).
A = liz, C = ann ;
A = liz, C = pat.

?- uncle(U, C).
U = bob, C = sue.

?- uncle(U, ann).
false.
```

(If the toplevel waits after an answer, press Enter to stop or ; for more.)

Key Takeaways

- New relations are built by chaining old ones: `uncle` = `sibling` of a parent, plus `male`.
- Put the most restrictive goals first so later goals work on known values.
- Gender facts turn one family relation into two (`uncle` and `aunt`).
- Define a relation once (`sibling`) and reuse it everywhere.

Mnemonic

“Parent’s sibling: boy = uncle, girl = aunt.” (P-S-G: **P**arent → **S**ibling → **G**ender.)

Summary of All Problems

#	Predicate	Technique	Base case
1	<code>fact/2</code>	Counting down with <code>is</code>	<code>fact(0,1)</code>
2	<code>sum_even/2</code>	List recursion with two guarded rules	<code>sum_even([],0)</code>
3	<code>palindrome/1</code>	List equals its reverse (accumulator)	<code>reverse_acc([],A,A)</code>
4	<code>max_list/2</code>	Head versus maximum of the tail	one-element list
5	<code>list_length/2</code>	Tail length plus one	<code>[]</code>
6	<code>reverse_list/2</code>	Accumulator	<code>reverse_acc([],A,A)</code>
7	<code>ancestor/2</code> , <code>descendant/2</code>	Recursion over relationships	direct <code>parent</code>
8	<code>eval/2</code>	Recursion over a term (tree)	<code>number(N)</code>
9	<code>count_elem/3</code>	Repeated variable in head, <code>\=</code> guard	<code>[]</code>
10	<code>find_siblings/2</code>	<code>findall/3</code> and <code>sort/2</code>	(none: not recursive)
11	<code>nth_element/3</code>	Counter goes down as list shrinks	position 1
12	<code>sorted/1</code>	Compare neighbours with <code>[X,Y T]</code>	<code>[]</code> and <code>[_]</code>
13	<code>is_prime/1</code>	<code>\+</code> and a divisor search	<code>D*D > N</code>
14	<code>fib/2</code>	Tree recursion	<code>fib(0,0)</code> , <code>fib(1,1)</code>
15	<code>uncle/2</code> , <code>aunt/2</code>	Combine <code>parent</code> , <code>sibling</code> , <code>gender</code>	(none: not recursive)

Definition: The five habits behind every solution

1. **Base case first**, recursive case second.
2. **Shrink the problem** in every recursive call.
3. Use **guards** (`N > 0`, `H \= E`, ...) so that clauses do not overlap.
4. **Calculate with `is`**, compare with `==`, `<`, `<=`; match with unification.
5. **Trust backtracking**: when a goal fails, Prolog automatically tries the next possibility.